CARETAG

Capstone Project Report

MID SEMESTER EVALUATION

Submitted by:

102303858, Kinjal

102303855, Guneev Taneja

102303476, Bhavishya Singla

102303876, Manya Sharma

102319027, Anureet

BE Fourth Year, CoE/CoSE

CPG No: 330

Under the Mentorship of

Dr. Kapil Tomar

Associate Professor



Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

ABSTRACT

Child and elderly safety monitoring systems rely predominantly on static, rule-based geofencing fixed geographic boundaries configured manually by caregivers. These systems have no awareness of an individual's daily movement patterns, generate alerts based purely on boundary crossing, and provide no quantified measure of how abnormal a situation is. CareTag addresses these gaps through a two-component system: an ML pipeline and an interactive geofence dashboard. The ML pipeline processes raw GPS trajectory data from the Microsoft GeoLife dataset, extracts stay points using the Li et al. distance-time algorithm, clusters them into named locations via K-Means, and trains a dual-input LSTM to model each individual's daily location routine across 30-minute time slots. Anomaly scoring combines a spatial score (normalized Haversine distance to the expected cluster centroid) and a temporal score (1 − softmax probability from LSTM output) into a weighted deviation score, with values above 0.6 flagged as anomalies. The geofence dashboard provides a browser-based interface built in React and Leaflet where caregivers define safe zones on a map and observe a simulated GPS device moving between them in real time. The simulation engine generates GPS packets every three seconds, computes deviation scores client-side, and fires alert banners when anomalous movement is detected. The system operates fully offline via localStorage and optionally connects to a .NET 8 backend with PostgreSQL and a FastAPI scoring microservice over SignalR. CareTag demonstrates that behavioural routine modeling from GPS history produces more contextually accurate safety alerts than static geofencing alone.

Keywords: GPS trajectory analysis, stay point detection, LSTM, anomaly detection, geofencing, behavioural monitoring, GeoLife

DECLARATION

We hereby declare that the design principles and working prototype model of the project entitled CareTag is an authentic record of our own work carried out in the Computer Science and Engineering Department, TIET, Patiala, under the guidance of Dr. Kaplil Tomar and during 6th semester (2026).

Date:

Roll No.	Name	Signature
102303858	KINJAL	Kinjal
102303855	GUNEEV	Guneev
102303476	BHAVISHYA	Bhavishya
102303876	MANYA	Manya
102319027	ANUREET	Anureet

Counter Signed By:

Faculty Mentor: Dr. Kapil Tomar

Dr. _____

CSED,

TIET, Patiala

ACKNOWLEDGEMENT

We would like to express our thanks to our mentor(s) Dr. Kapil Tomar. He has been of great help in our venture and an indispensable resource of technical knowledge. He is truly an amazing mentor to have.

We are also thankful to Dr. Ashima Singh, Head, Computer Science and Engineering Department, the entire faculty and staff of the Computer Science and Engineering Department, and also our friends who devoted their valuable time and helped us in all possible ways towards successful completion of this project. We thank all those who have contributed either directly or indirectly towards this project.

Lastly, we would also like to thank our families for their unyielding love and encouragement. They always wanted the best for us and we admire their determination and sacrifice.

Date: 22/08/2026

Roll No.	Name	Signature
102303858	Kinjal	Kinjal
102303855	Guneev	Guneev
102303476	Bhavishya	Bhavishya
102303876	Manya	Manya
102319027	Anureet	Anureet

TABLE OF CONTENTS

ABSTRACT……2
DECLARATION……3
ACKNOWLEDGEMENT……..4
LIST OF FIGURES……..5
LIST OF TABLES……..6
LIST OF ABBREVIATIONS……..7

CHAPTER…….

1. **Introduction**
   - 1.1 Project Overview (3-4 pages)
   - 1.2 Need Analysis (1 page- mentioning the significance of the work)
   - 1.3 Research Gaps (Identify and explain at least five research gaps with references)
   - 1.4 Problem Definition and Scope
   - 1.5 Assumptions and Constraints
   - 1.6 Standards
   - 1.7 Approved Objectives (Objectives approved in Proposal Evaluation by panel)
   - 1.8 Methodology
   - 1.9 Project Outcomes and Deliverables
   - 1.10 Novelty of Work
2. **Requirement Analysis**
   - 2.1 Literature Survey
     - 2.1.1 Theory Associated With Problem Area
     - 2.1.2 Existing Systems and Solutions
     - 2.1.3 Research Findings for Existing Literature
     - 2.1.4 Problem Identified
     - 2.1.5 Survey of Tools and Technologies Used
   - 2.2 Software Requirement Specification
     - 2.2.1 Introduction
       - 2.2.1.1 Purpose
       - 2.2.1.2 Intended Audience and Reading Suggestions
       - 2.2.1.3 Project Scope
     - 2.2.2 Overall Description
       - 2.2.2.1 Product Perspective
       - 2.2.2.2 Product Features
     - 2.2.3 External Interface Requirements
       - 2.2.3.1 User Interfaces

2.2.3.2 Hardware Interfaces
2.2.3.3 Software Interfaces
2.2.4 Other Non-functional Requirements
2.2.4.1 Performance Requirements
2.2.4.2 Safety Requirements
2.2.4.3 Security Requirements
2.3 Cost Analysis
2.4 Risk Analysis

3. Methodology Adopted

3.1 Investigative Techniques (Justify the selected investigative technique for your project, 2-3 pages)
3.2 Proposed Solution (2-3 pages)
3.3 Work Breakdown Structure (along with a discussion on workable modules/products)
3.4 Tools and Technology

4. Design Specifications (Sub-sections may vary according to the applicability of diagrams for student projects)

4.1 System Architecture (Eg: Block Diagram /Technology Stack/ MVC/ Tier architecture whichever suits the project)
4.2 Design Level Diagrams
4.3 User Interface Diagrams
4.4 Snapshots of Working Prototype (along with a step-by-step discussion of the working prototype)

5. Conclusions and Future Scope

5.1 Work Accomplished (Discussion w.r.t. the approved objectives)
5.2 Conclusions
5.3 Environmental (/ Economic/ Social) Benefits
5.4 Future Work Plan

APPENDIX A: References

APPENDIX B: Plagiarism Report

*Note: Diagrams should have a detailed explanation. Do refer to figure/table numbers in the running text also.

LIST OF TABLES

Table No.	Caption	Page No.
Table 1	Commercial Systems	11
Table 2	Research Findings for Existing Literature	12
Table 3	Survey of Tools and Technologies Used	14
Table 4	Intended Audience and Reading Suggestions	15
Table 5	Hardware Interfaces	17
Table 6	Software Interfaces	17
Table 7	Performance Requirements	18
Table 8	Cost Analysis	18
Table 9	Risk Analysis	19

LIST OF FIGURES

Figure No.	Caption	Page No.
Figure 4.1	CareTag three-tier system architecture	31
Figure 4.2	CareTag use case diagram	32
Figure 4.3	Sequence diagram — anomaly alert flow	32
Figure 4.4	Activity diagram — geofence breach and anomaly detection	33
Figure 4.5	UI wireframe — geofence management screen	34
Figure 4.6	Basic overview of CareTag	34
Figure 4.7	Simulating the tracking of person	35

LIST OF ABBREVIATIONS

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACM	Association for Computing Machinery
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CPG	Capstone Project Group
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
EPSG	European Petroleum Survey Group
GIS	Geographic Information System
GPU	Graphics Processing Unit
GSM	Global System for Mobile Communications
HMM	Hidden Markov Model
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
I²C	Inter-Integrated Circuit
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IMU	Inertial Measurement Unit
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MCU	Microcontroller Unit
ML	Machine Learning
MVC	Model-View-Controller
NCRB	National Crime Records Bureau
PEP	Python Enhancement Proposal
PII	Personally Identifiable Information
RAM	Random Access Memory
REST	Representational State Transfer
RNN	Recurrent Neural Network
SPA	Single Page Application
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
WBS	Work Breakdown Structure
WCSS	Within-Cluster Sum of Squares
WGS	World Geodetic System

Chapter 1, INTRODUCTION

1.1 Project Overview

The global population of individuals aged 65 and above is projected to exceed 1.5 billion by 2050, with an estimated 50 million people currently living with dementia (World Health Organization, 2023). Simultaneously, child safety remains a persistent concern, the National Crime Records Bureau reported over 1.48 lakh crimes against children in India in 2019 alone. For both demographics, location-based safety monitoring is increasingly critical. However, the dominant technological approach to this problem, static geofencing, is fundamentally reactive and context-blind. A caregiver manually draws a boundary on a map; the system alerts when the boundary is crossed. This approach has three structural failures: it has no knowledge of the monitored individual's routine, it provides no temporal context (being at a park at 4 PM is normal; at 2 AM it is not), and it generates binary alerts with no information about the severity or distance of the deviation.

CareTag is a two-component system designed to address these failures. The first component is an ML pipeline that learns behavioral routines from historical GPS trajectory data. The second is a browser-based geofence dashboard that allows caregivers to define safe zones, run GPS simulations, and receive anomaly-scored real-time alerts.

ML Pipeline. The pipeline processes raw GPS trajectory files in the `.plt` format from the Microsoft GeoLife dataset, a publicly available dataset comprising 182 users and 17,621 trajectories collected in Beijing between 2007 and 2012. Processing proceeds through three stages:

Stage 1, Stay Point Extraction: Raw GPS coordinates are parsed and filtered to identify locations where a person remained within 200 metres for at least 20 minutes. This algorithm, formalized by Li et al. (2008), reduces thousands of GPS points to a manageable set of semantically meaningful stops. The output is a set of stay points, each with a centroid coordinate and a timestamp range.

Stage 2, Location Clustering and Routine Modeling: Stay points are clustered using K-Means (evaluated up to K = 10 via the elbow method) to produce a set of canonical named locations, Home, Work, School, and so on, represented as cluster centroids. Daily location visits are then encoded into 30-minute time slots and used to train a dual-input LSTM. One input stream encodes the current

time slot; the other encodes recent cluster visit history. The LSTM learns to predict which cluster a person is expected to be at, given the time of day and their recent movement history.

Stage 3, Anomaly Scoring: For any incoming GPS coordinate, an anomaly score is computed as the weighted combination of two signals: (1) a spatial score, the normalized Haversine distance between the GPS coordinate and the expected cluster centroid; and (2) a temporal score, computed as 1 − P(actual cluster | context), where P is the softmax probability output of the LSTM. The combined deviation score is continuous on [0, 1]. Coordinates scoring above 0.6 are flagged as anomalous.

Geofence Dashboard. The dashboard is a React 18 single-page application using Leaflet for map rendering. Caregivers interact with an interactive map to place, name, and manage circular geofences at any geographic location. A built-in GPS simulation engine (`simulation.js`) generates a virtual device that moves between saved geofences in realistic phases, STAY (stationary within a geofence), COMMUTE (transitioning between geofences), and ANOMALY (random wander away from all geofences). GPS packets are generated every three seconds with jitter, simulated speed, and battery telemetry. When the deviation score exceeds the threshold, an alert banner fires with the score and distance to the nearest geofence. The dashboard operates fully offline via localStorage, with an optional backend consisting of a .NET 8 REST API, PostgreSQL database, and a FastAPI Python anomaly scoring microservice connected via SignalR WebSocket for real-time telemetry push.

The system is developed primarily in Python (ML pipeline), React (frontend), and .NET 8 (backend), with Docker Compose managing the optional multi-service deployment. The trained models, a K-Means `.joblib` file and a PyTorch LSTM `.pth` file, are pre-built from GeoLife user 000 and deployable to the scoring microservice without retraining.

CareTag differs from existing commercial systems such as Google Family Link, Life360, and Apple Find My in that it does not treat safety monitoring as a boundary-crossing problem. It treats it as a behavioral deviation problem, where the reference is the individual's own learned routine, not a manually drawn polygon.

1.2 Need Analysis

Demographic Context. India's elderly population (60+) is expected to reach 340 million by 2050 (United Nations, 2022). Dementia affects approximately 8.8 million Indians currently, with wandering behavior reported in 17–40% of cases (Alzheimer's and Related Disorders Society of India, 2020). For children, the gap between unsupervised outdoor hours and parental monitoring

capability has widened with dual-income households. Both groups share a common need: real-time, intelligent location awareness for caregivers.

Inadequacy of Existing Solutions. Current commercial solutions are GPS ping systems with manual boundary configuration. They are incapable of distinguishing a new-but-safe location visit from a genuinely anomalous one. A child who regularly visits a friend's house after school will trigger false alerts every time, alert fatigue causes caregivers to disable or ignore notifications. An elderly person with dementia who begins deviating from routine in subtle ways, slightly later arrivals, new detours, produces no signal at all until they cross a hard boundary.

Significance of Behavioral Modeling. The shift from boundary-based to routine-based monitoring has direct safety implications. Research by Liao et al. (2007) showed that temporal context improves location prediction accuracy significantly over spatial-only models. A system that understands that a person should be home by 6 PM on weekdays can flag a 7 PM absence as suspicious, even if the person is still within any manually drawn geofence. This contextual intelligence is what CareTag provides.

Quantified Alerting. Binary alerts (triggered / not triggered) give caregivers no information for prioritization. A continuous deviation score on [0, 1] allows caregivers to distinguish a minor boundary excursion from a severe anomaly, enabling proportionate response. This is particularly important in care facilities managing multiple individuals simultaneously.

Technical Accessibility. CareTag operates fully in a browser without backend infrastructure, making it accessible to caregivers and researchers without DevOps expertise. The ML pipeline trains on open data. The hardware layer (ESP32, Neo-6M GPS, SIM800L) costs under ₹2,500 for a physical prototype, making the full system viable for deployment in resource-constrained environments.

1.3 Research Gaps

Gap 1, Static Geofences Cannot Model Individual Routines (Li et al., 2008; Mazimpaka & Timpf, 2016)

All major commercial GPS monitoring systems use manually configured, static geofences. These zones reflect caregiver intent, not the monitored individual's actual behavioral pattern. A person who visits five different locations throughout a typical week requires five separate geofences configured correctly, any visit outside these will alert, including routine visits to new-but-safe

locations. Li et al. (2008) demonstrated that stay point extraction from GPS history captures semantically meaningful locations automatically; this capability is absent in all commercial monitoring tools reviewed. Mazimpaka & Timpf (2016) identified this as a central limitation of applied trajectory systems.

Gap 2, No Temporal Context in Location Alerts (Liao et al., 2007; Zheng et al., 2009)

Existing geofencing systems evaluate location purely spatially, is the device inside or outside the boundary? They have no model of what time it is or whether the current location is appropriate for the current time. Liao et al. (2007) showed that temporal conditioning significantly improves location prediction accuracy. Zheng et al. (2009) demonstrated that sequential GPS visit patterns predict future locations effectively. Neither insight is applied in any commercial monitoring system. CareTag integrates time-slot encoding directly into the LSTM input to address this gap.

Gap 3, Binary Alerting Provides No Severity Information (Ji et al., 2020)

Commercial systems produce binary alerts: the geofence was crossed, or it was not. Caregivers managing multiple individuals receive undifferentiated notifications with no way to prioritize response. Ji et al. (2020) showed that combining spatial distance and semantic trajectory deviation produces a graded anomaly signal that more accurately reflects the severity of the situation. CareTag implements a continuous deviation score [0, 1] derived from both spatial (Haversine) and temporal (LSTM softmax) signals, directly filling this gap.

Gap 4, Personalized Geofences Are Not Constructed Automatically (Lin et al., 2018)

Lin et al. (2018) proposed constructing personalized geofences for elderly individuals by mining their historical GPS trajectories. Their method produced irregular polygon boundaries adapted to each individual's routine movement graph, significantly outperforming manually configured rectangular or circular geofences. No commercial system implements trajectory-derived geofence construction. CareTag uses K-Means clustering on extracted stay points to derive cluster centroids that serve as geofence anchors, implementing the principle of data-driven boundary construction from GPS history.

Gap 5, Real-Time Anomaly Scoring Requires Batch Retraining in Existing Academic Systems (Mazimpaka & Timpf, 2016; Hochreiter & Schmidhuber, 1997)

Mazimpaka & Timpf (2016) identified that most GPS trajectory anomaly detection systems operate in batch mode, they analyze a completed trajectory after the fact, rather than scoring each GPS packet in real time. This renders them unsuitable for live safety monitoring where sub-second alert

latency matters. CareTag's anomaly scorer computes the deviation score for each incoming GPS coordinate independently, using the pre-trained LSTM for inference without requiring retraining or batch context. Client-side Haversine scoring ensures this works even without backend connectivity.

Gap 6, No Offline-Capable, Caregiver-Facing Dashboard Exists for ML-Based Monitoring

Existing academic work on GPS behavioral anomaly detection publishes results as static analyses or server-dependent systems. No reviewed system provides a caregiver-facing interactive interface that works without backend infrastructure. CareTag's dashboard operates fully in the browser via localStorage, enabling deployment in environments with no cloud access. This is a practical gap in the literature not addressed by any prior system.

1.4 Problem Definition and Scope

Problem Statement. Existing GPS-based safety monitoring systems for children and elderly individuals rely on static, manually configured geofences and binary alerting. These systems are incapable of modeling individual behavioral routines, providing no temporal context for location events and no quantified measure of anomaly severity. As a result, they generate false positives for routine-but-unregistered location visits and fail to detect subtle behavioral deviations that precede safety incidents.

Scope.

In scope:

- GPS trajectory data processing using the GeoLife dataset (Microsoft, 2012)
- Stay point extraction, K-Means location clustering, and LSTM routine modeling
- Real-time anomaly scoring (spatial + temporal weighted deviation)
- Browser-based geofence management dashboard with GPS simulation engine
- Optional backend: .NET 8 REST API, PostgreSQL, FastAPI scoring microservice, SignalR
- Offline operation via localStorage

Out of scope:

- Physical hardware integration (ESP32, Neo-6M GPS, SIM800L GSM) in current prototype
- Live GPS device communication and real-world field testing
- Multi-user authentication and access control

- Mobile application development
- Integration with emergency services or external alert systems

1.5 Assumptions and Constraints

1.5.1 Assumptions

S. No.	Assumptions
1	It is assumed that the monitored individual's GPS trajectory data is available in GeoLife `.plt` format or a structurally compatible format. The system has been designed and validated using the Microsoft GeoLife dataset (182 users, 17,621 trajectories). For deployment on a new user, a minimum of 20 days of trajectory data is assumed to be available for LSTM training to converge meaningfully.
2	It is assumed that the monitored individual follows a sufficiently regular daily routine, visiting a consistent set of locations (home, school, workplace, etc.) across weekdays. The LSTM routine model relies on temporal patterns in location sequences; highly irregular movement patterns may reduce model accuracy. It is further assumed that the individual's routine remains stable over the training period.
3	A number of factors regarding GPS signal availability are assumed: GPS signal is assumed to be available in outdoor environments. Indoor positioning is not modeled in the current system. It is assumed that the GPS module (Neo-6M or equivalent) achieves positioning accuracy within 5 metres in open outdoor conditions, consistent with standard civilian GPS specifications.
4	It is assumed that the caregiver operating the dashboard has access to a modern web browser (Chrome 90+, Firefox 88+, or equivalent) and a stable internet connection for map tile rendering. All geofence logic, simulation, and anomaly scoring run client-side and do not require internet connectivity beyond tile loading. It is assumed the user is familiar with basic web application interaction (clicking, form input).
5	It is assumed that the anomaly threshold of 0.6 is appropriate as a default starting point for flagging deviation. This value was selected based on the weighted combination of spatial and temporal scores during testing on GeoLife user 000 data. Per-deployment threshold calibration may be required for individuals with atypical movement patterns.

1.5.2 Constraints

S. No.	Constraints
1	The ML pipeline is trained on GeoLife user 000 data (50 trajectory files). Generalization to other users requires retraining the K-Means and LSTM models on that individual's GPS history. The pre-built models shipped in `outputs/` are not universally applicable without retraining.
2	The LSTM model performs CPU-only inference. GPU acceleration is not implemented in the current prototype. Inference latency on a standard laptop CPU is bounded at under 50ms per single input, and under 200ms per request via the FastAPI scoring microservice.
3	The dashboard uses OpenStreetMap tiles for map rendering. Map tile loading requires an active internet connection. All other dashboard functionality, geofence CRUD, simulation, anomaly scoring, and alerts, operates fully offline via localStorage without any external dependency.
4	The localStorage-based offline mode has a practical limit of approximately 50 saved geofences before browser storage quota is approached. For larger deployments or persistent multi-session storage, migration to the PostgreSQL backend via Docker Compose is required.
5	The full backend stack (.NET 8 API + FastAPI scorer + PostgreSQL) requires Docker Desktop installed with a minimum of 4 GB RAM allocated. The `docker compose up --build` command must be executed from the `geofence-dashboard/backend/` directory. Environments without Docker support must operate in standalone localStorage mode.

1.6 Standards

Standard / Protocol	Governing Body	Application in CareTag
IEEE 802.11 (Wi-Fi)	IEEE	Dashboard-to-backend communication in local network deployments
HTTPS / TLS 1.3	IETF RFC 8446	Secure API communication in production deployment
REST (HTTP/1.1)	W3C / IETF	.NET 8 backend API design
WebSocket (RFC 6455)	IETF	SignalR real-time telemetry channel
GeoJSON (RFC 7946)	IETF	Geofence coordinate serialization
WGS 84	EPSG:4326	GPS coordinate reference system (Latitude/Longitude)
Haversine Formula	Spherical geometry	Great-circle distance calculation for anomaly scoring
OpenAPI 3.0	OpenAPI Initiative	.NET 8 API documentation (Swagger)
PEP 8	Python Software Foundation	Python codebase style standard
ISO/IEC 25010	ISO/IEC	Software quality model, basis for non-functional requirements

1.7 Approved Objectives

The following objectives were approved by the project evaluation panel at the time of proposal submission:

1. To extract stay points from raw GPS trajectory data using the distance-time threshold algorithm (Li et al., 2008) and cluster them into named canonical locations using K-Means.
2. To design and train a dual-input LSTM model that predicts the expected cluster for a monitored individual at any given time, conditioned on time-slot encoding and recent location history.
3. To implement a real-time anomaly scorer that computes a continuous deviation score [0, 1] by combining normalized Haversine spatial distance and LSTM temporal probability output, flagging scores above 0.6 as anomalous.
4. To develop a browser-based geofence management dashboard in React and Leaflet that allows caregivers to define, edit, and delete named safe zones on an interactive map.
5. To implement a GPS simulation engine that generates realistic multi-phase device movement (STAY, COMMUTE, ANOMALY) between saved geofences, with real-time telemetry display and anomaly alert banners.
6. To design the dashboard for full offline operation via localStorage, with optional backend integration using .NET 8, PostgreSQL, FastAPI, and SignalR.

1.8 Methodology

CareTag follows a pipeline-based methodology with two parallel tracks, ML development and dashboard development, converging at the anomaly scoring interface.

Track 1, ML Pipeline

Raw GPS data in .plt format is loaded from the GeoLife dataset using a custom parser (data_loader.py). Stay points are extracted by applying the Li et al. (2008) algorithm with a 200m spatial threshold and 20-minute temporal threshold. The resulting stay points are clustered into K groups using K-Means, with the optimal K selected via the elbow method on within-cluster sum of squares (WCSS). Cluster centroids are saved as cluster_centers.csv.

Daily location sequences are encoded into 30-minute time slots and saved as training sequences (training_sequences.npz). A dual-input PyTorch LSTM is trained on these sequences, one input branch takes a one-hot time slot vector, the other takes recent cluster history. The model outputs a softmax probability distribution over clusters. Trained weights are saved as lstm_routine_model.pth.

The anomaly scorer accepts a raw GPS coordinate and returns a deviation score by computing the Haversine distance to the predicted cluster centroid (spatial score), retrieving the softmax probability of the actual cluster from the LSTM (temporal score), and combining them as: deviation $= \alpha \times$ spatial_score $+ (1 - \alpha) \times$ temporal_score, where $\alpha = 0.5$ by default.

Track 2, Dashboard

The React dashboard is initialized with Create React App. Leaflet is integrated for map rendering via **react-leaflet**. Geofences are stored in localStorage (offline) or via the .NET REST API (online). The simulation engine (simulation.js) implements a finite state machine with three phases. GPS packets are generated at 3-second intervals using random walk with configurable jitter. Deviation scores are computed client-side using the Haversine formula against saved geofence centroids. SignalR integration (signalr.js) enables WebSocket-based telemetry push from the backend when available.

The optional backend is dockerized using Docker Compose: the .NET 8 API serves REST endpoints and hosts the SignalR hub; the FastAPI scorer loads the trained models and exposes a /score endpoint; PostgreSQL provides persistent storage.

1.9 Project Outcomes and Deliverables

Deliverable	Description	Format
Trained K-Means model	Cluster model for GeoLife user 000	`kmeans_model.joblib`
Trained LSTM model	Routine prediction model	`lstm_routine_model.pth`
Cluster centroids	Named location anchors (lat/lon)	`cluster_centers.csv`
Labeled stay points	67 stay points with cluster assignments	`labeled_stay_points.csv`
ML pipeline codebase	End-to-end Python pipeline with CLI	GitHub repository
Geofence Dashboard	React + Leaflet SPA with simulation engine	Deployed localhost / GitHub Pages
Backend services	.NET API + FastAPI scorer + Docker Compose	Containerized, GitHub repository
Project Report	Full CPG report with all sections	DOCX / PDF
Plagiarism Report	Turnitin similarity report	PDF

Quantitative Outcomes

- Stay point extraction: 67 stay points extracted from 50 GeoLife trajectory files for user 000
- Cluster model: K-Means with optimal K evaluated up to 10
- Simulation fidelity: GPS packets every 3 seconds with speed, jitter, battery, and phase telemetry
- Anomaly threshold: deviation score > 0.6 flags anomaly
- Offline capability: full dashboard operation without any backend dependency

1.10 Novelty of Work

CareTag's contribution is distinct from both existing commercial systems and the academic literature on three specific axes:

1. Joint Spatial-Temporal Anomaly Scoring from Learned Routines
Existing commercial systems score anomalies spatially only (distance from a fixed boundary). Academic systems reviewed either focus on spatial clustering or temporal sequence modeling in isolation. CareTag combines both into a single weighted deviation score derived from a live LSTM inference, producing a more contextually accurate signal than either approach alone. Ji et al. (2020) demonstrated the advantage of combining spatial and semantic signals; CareTag applies the same principle in a safety monitoring context with an explicitly temporal model.

2. Trajectory-Derived Geofences as LSTM Cluster Anchors
Lin et al. (2018) proposed constructing personalized geofences from GPS trajectory mining. CareTag operationalizes this in a real-time context: the K-Means cluster centroids derived from stay point extraction serve as the geofence anchor points, making every geofence a data-driven construct

rather than a manual one. This is the first system reviewed that directly connects trajectory-mined clusters to caregiver-visible safe zones on an interactive map.

3. Offline-First Caregiver Dashboard with ML-Backed Simulation

No prior academic work reviewed provides a caregiver-facing interface that (a) integrates ML-based anomaly scoring, (b) includes a realistic GPS simulation engine for testing, and (c) operates fully in a browser without backend infrastructure. The localStorage-first, backend-optional architecture makes CareTag deployable in resource-constrained environments where cloud infrastructure is unavailable, a practical contribution not present in any comparable system.

Chapter 2, Need Analysis

2.1 Literature Survey

2.1.1 Theory Associated with Problem Area

GPS-based behavioral monitoring operates at the intersection of trajectory data mining, sequence modeling, and anomaly detection. The foundational problem is distinguishing *routine* movement from anomalous movement for a specific individual a task that requires modeling both spatial and temporal patterns jointly.

Stay Point Detection. A stay point is a geographic region where a person remains for a meaningful duration. Li et al. (2008) formalized this as a point cluster where displacement between consecutive GPS readings stays below a spatial threshold (typically 200m) for a minimum time window (20 minutes). Stay points act as a data reduction step compressing thousands of raw GPS coordinates into a manageable set of semantically meaningful locations (Home, Work, School).

Location Clustering. Once stay points are extracted, similar geographic locations across multiple days must be grouped. K-Means clustering on GPS coordinates (latitude, longitude) is the standard approach, with the number of clusters k determined via the elbow method on inertia. Each cluster centroid represents a canonical location in the person's routine.

Sequential Routine Modeling. Daily routines follow temporal structure, a person is at home in early morning, at school mid-day, back home by evening. Long Short-Term Memory (LSTM) networks are suited for this because they capture long-range dependencies in sequential data. A dual-input LSTM, conditioned on both the encoded time slot and recent location history, can predict the expected cluster at any given time.

Anomaly Scoring. Anomalies are quantified as deviation from the model's expectation. Two signals are combined: (1) spatial deviation, Haversine distance between the actual position and the expected cluster centroid, normalized to [0, 1]; (2) temporal deviation computed as `1 − P(actual_cluster | context)` from the LSTM's softmax output. A weighted combination of these two scores produces a final deviation score, with values above 0.6 flagged as anomalous.

Geofencing. A geofence is a virtual geographic boundary associated with a named real-world location. Traditional geofencing is purely rule-based (inside/outside a polygon). CareTag extends this by integrating geofences with learned behavioural models anomalies are detected not just when

a person leaves a defined zone, but when their location is inconsistent with their historical routine at that time of day.

2.1.2 Existing Systems and Solutions

System	Approach	Limitation
Google Family Link	Static geofencing + device GPS polling	No behavioral learning; alert on exit only
Life360	Location sharing + manual place creation	Rule-based; no anomaly modeling
Apple Find My	Passive location broadcast	No alerts; no routine analysis
Samsung SmartThings	IoT presence detection via Wi-Fi/BLE	Not GPS-based; short range

All four are fundamentally reactive systems. They alert when a device crosses a manually configured boundary. None model what the expected behaviour *at a given time should be* for a specific individual.

Academic Systems

Zheng et al. (2010) used GeoLife trajectory data to classify transportation modes (walking, driving, cycling) using GPS features fed into decision trees and conditional random fields. This established GeoLife as a standard benchmark for GPS behavioural analysis.

Liao et al. (2007) built a hierarchical Markov model for recognizing activities from GPS traces, showing that temporal context significantly improves location prediction over purely spatial models.

Jiang et al. (2013) proposed a hybrid approach combining stay-point extraction with hidden Markov models (HMM) for routine discovery, achieving location prediction accuracy of ~78% on personal trajectory datasets.

Mazimpaka & Timpf (2016) reviewed trajectory data mining comprehensively, identifying that most existing approaches require batch retraining rather than supporting real-time scoring a gap CareTag addresses through its anomaly scoring module.

2.1.3 Research Findings for Existing Literature

S. No.	Roll Number	Name	Paper Title	Tools/ Technology	Findings	Citation
1	102303855	Guneev	Mining User Similarity Based on Location History	GPS trajectories, K-Means, stay point algorithm	Proposed the stay point detection algorithm (distance + time threshold) now standard in GPS behavioral analysis.	Li, Q., Zheng, Y., Xie, X., Chen, Y., Liu, W., & Ma, W. Y. (2008). *ACM SIGSPATIAL GIS.*

2			Understanding Mobility Based on GPS Data	GeoLife dataset, GPS, clustering	Established the GeoLife dataset as a benchmark; showed that stay points capture semantically meaningful locations from raw trajectories	Zheng, Y., Li, Q., Chen, Y., Xie, X., & Ma, W. Y. (2008). *UbiComp 2008.* ACM.
3			Mining Interesting Locations and Travel Sequences from GPS Trajectories	GPS, hierarchical graph, sequence mining	Demonstrated that sequential patterns in GPS location visits can predict future locations; underpins LSTM-based routine modeling	Zheng, Y., Zhang, L., Xie, X., & Ma, W. Y. (2009). *WWW 2009.* ACM.
4			Long Short-Term Memory	RNN, LSTM, backpropagation through time	Introduced the LSTM architecture; showed it solves the vanishing gradient problem in long sequences the theoretical basis for CareTag's routine model	Hochreiter, S., & Schmidhuber, J. (1997). *Neural Computation, 9*(8), 1735–1780.
5			Learning and Inferring Transportation Routines	GPS, HMM, Bayesian inference	Built routine models from GPS traces using HMMs; showed temporal context (time of day) significantly improves location prediction over spatial-only models	Liao, L., Patterson, D. J., Fox, D., & Kautz, H. (2007). *Artificial Intelligence, 171*(5–6), 311–331.
6			Trajectory Data Mining: A Review of	GPS, GIS, clustering, sequence modeling	Reviewed state-of-art trajectory mining approaches;	Mazimpaka, J. D., & Timpf, S. (2016). *Journal of Spatial Information Science, 13,* 61–99.

			Methods and Applications		identified gap: most systems require batch retraining and lack real-time anomaly scoring	
7	102303855	Gunee v	A Method for LSTM-Based Trajectory Modeling and Abnormal Trajectory Detection	LSTM, Seq2Seq, spatiotemporal trajectory features, IEEE Access	Seq2Seq LSTM learns normal trajectories without manual feature extraction; combining spatial and semantic information outperforms spatial-only anomaly detection.	Ji et al. (2020). *IEEE Access*, *8*, 104063–104073. https://doi.org/10.1109/ACCESS.2020.2997967
8			GPS Trajectories Based Personalized Safe Geofence for Elders with Dementia	GPS trajectory mining, graph-based movement modeling, geofencing, data mining	Mining historical GPS trajectories to build personalized geofences outperforms static manually-configured boundaries; individual routine patterns must inform safe zone construction.	Lin et al. (2018). *IEEE SmartWorld/SCALCOM/UIC/ATC* (pp. 505–514). https://doi.org/10.1109/SmartWorld.2018.00111

2.1.4 Problem Identified

Existing safety monitoring systems have three concrete gaps that CareTag addresses:

Gap 1, No personalized routine learning. Geofences are configured manually by a caregiver. The system has no knowledge of the monitored person's typical daily patterns. A child who always goes to a friend's house after school will generate alerts every time even though the visit is routine.

Gap 2, No temporal context in alerts. Existing systems detect *where* a person is, not *whether that location is appropriate for this time of day*. Being at a park is normal at 4 PM and anomalous at 2 AM, but a static geofence treats both identically.

Gap 3, No quantified deviation. Alerts from commercial systems are binary (triggered / not triggered). Caregivers receive no information about *how far* from normal a situation is a person 50m outside their geofence and a person 5km away both generate the same alert.

CareTag addresses all three: it learns individual routines from GPS history, scores anomalies with temporal context from an LSTM, and outputs a continuous deviation score alongside binary alerts.

2.1.5 Survey of Tools and Technologies Used

Tool / Technology	Version	Role in Project
Python	3.10+	ML pipeline development
PyTorch	2.x	LSTM model implementation and training
scikit-learn	Latest stable	K-Means clustering, preprocessing
pandas	Latest stable	GPS data parsing and feature engineering
GeoLife Dataset	v1.3	Training data (182 users, 17,621 trajectories)
React	18	Frontend dashboard
Leaflet.js	Latest stable	Interactive map rendering
.NET	8	Backend REST API
PostgreSQL	15.x	Persistent geofence and telemetry storage
FastAPI	Latest stable	Python-based anomaly scoring microservice
SignalR	.NET integrated	WebSocket-based real-time telemetry push
Docker Compose	Latest stable	Multi-service container orchestration

2.2 Software Requirement Specification

2.2.1 Introduction

2.2.1.1 Purpose

This Software Requirement Specification (SRS) defines the functional and non-functional requirements for CareTag, a GPS behavioral monitoring system combining machine learning-based routine modeling with an interactive geofence management dashboard. The document serves as the basis for system design, implementation, and testing, and is intended to ensure all components are built to a consistent and verifiable specification.

2.2.1.2 Intended Audience and Reading Suggestions

Reader	Relevant Sections

Developers	All sections, particularly 2.2.2, 2.2.3
Evaluators / Faculty	2.1, 2.2.1, 2.2.2, 2.2.4, 2.3, 2.4
End Users (Caregivers)	2.2.1.3, 2.2.2.2, 2.2.3.1
System Administrators	2.2.3.2, 2.2.3.3, 2.2.4

2.2.1.3 Project Scope

CareTag covers two functional scopes:

ML Engine (CareTag): Processes raw GPS trajectory files from the GeoLife dataset, extracts stay points, clusters them into named locations, trains a dual-input LSTM routine model, and produces real-time anomaly scores for new GPS coordinates.

Geofence Dashboard: A browser-based interface allowing caregivers to define named safe zones (geofences) on a map, trigger a GPS simulation engine that moves a virtual device between geofences, and receive anomaly alerts when the device deviates from safe zones. Supports offline operation via localStorage and optional backend persistence via Docker-composed services.

Out of scope: physical hardware integration (ESP32, GSM module), actual live GPS device communication, and multi-user authentication in the current version.

2.2.2 Overall Description

2.2.2.1 Product Perspective

CareTag is a standalone system with an optional backend extension. In its minimal form (frontend only), it runs entirely in a browser with no server dependency. In full deployment, it connects to a .NET REST API backed by PostgreSQL, with a FastAPI microservice handling ML-based anomaly scoring. The system is not a component of a larger existing product it is a self-contained research prototype.

The ML pipeline operates offline: training runs on stored GeoLife data, and the trained models (K-Means `.joblib` + LSTM `.pth`) are deployed to the scoring microservice. The dashboard can consume these scores over HTTP or compute simplified deviation scores locally using the Haversine formula.

2.2.2.2 Product Features

F1, Stay Point Extraction: Parse GeoLife `.plt` GPS trajectory files and identify locations where the user remained within 200m for at least 20 minutes.

F2, Location Clustering: Group extracted stay points into K meaningful clusters using K-Means (K evaluated up to 10 via elbow method). Output cluster centroids as named location anchors.

F3, LSTM Routine Modeling: Encode daily location visits into 30-minute time slots and train a dual-input LSTM, one input for time slot encoding, one for recent cluster history to predict the expected cluster at any time.

F4, Anomaly Scoring: For new GPS input, compute a deviation score = weighted sum of spatial score (normalized Haversine distance to expected cluster) and temporal score (1 − LSTM softmax probability). Flag as anomaly if score > 0.6.

F5, Geofence Management: Allow caregivers to place, name, resize, and delete circular geofences on an interactive Leaflet map. Persist geofences via localStorage or backend API.

F6, GPS Simulation Engine: Simulate a virtual device moving between saved geofences with realistic telemetry (speed, jitter, battery level). Phases: STAY → COMMUTE → occasional ANOMALY wander. GPS packets generated every 3 seconds.

F7, Real-Time Anomaly Alerts: Display alert banners with deviation score and distance when the simulated device exits safe zones. Push telemetry updates via SignalR when backend is running.

F8, Offline Mode: Full dashboard functionality (geofence CRUD + simulation + alerts) without any backend local Storage replaces API; deviation scoring runs client-side.

2.2.3 External Interface Requirements

2.2.3.1 User Interfaces

The dashboard provides a single-page application with the following UI components:

- **Map Panel (Primary):** Full-viewport interactive map (Leaflet + OpenStreetMap tiles). Caregivers click to drop a pin, enter a name, and save a geofence. Geofences render as labeled purple circles. A moving marker represents the simulated device.
- **Sidebar Panel:** Displays real-time telemetry current phase (STAY / COMMUTE / ANOMALY), speed (km/h), deviation score (0–1), nearest geofence name, and distance to it.
- **Alert Banner:** Full-width banner with red background, deviation score, and distance fires when score exceeds 0.6. Dismissible.
- **Control Bar:** Contains Start / Stop simulation button and geofence list with delete controls.

UI requirements:

- Responsive layout supporting 1280px+ desktop viewports
- Map tiles load from OpenStreetMap CDN (requires internet for tiles; all logic runs locally)
- No authentication UI in current scope

2.2.3.2 Hardware Interfaces

In the standalone simulation mode, no physical hardware is required. In full production deployment:

Hardware	Role	Interface
ESP32 Microcontroller	On-device GPS + sensor processing	USB serial (development); GSM (production)
Neo-6M GPS Module	Real GPS coordinate acquisition	UART to ESP32
SIM800L GSM Module	Cellular data transmission to backend	AT commands via UART
MPU-6050 IMU	Motion/fall data (future integration)	I^2C to ESP32

For the current software-only prototype, these interfaces are simulated by `simulation.js`.

2.2.3.3 Software Interfaces

Interface	Protocol	Description
Geofence REST API	HTTP/JSON (.NET 8)	CRUD operations on geofences stored in PostgreSQL
Anomaly Scorer API	HTTP/JSON (FastAPI)	Accepts GPS coordinates, returns deviation score
SignalR Hub	WebSocket (.NET SignalR)	Streams real-time telemetry packets to dashboard
OpenStreetMap Tiles	HTTPS	Satellite/street map tiles for Leaflet
localStorage	Browser API	Fallback persistence for geofences and simulation state
GeoLife Dataset	File I/O (.plt format)	Input to ML pipeline; not a runtime interface

2.2.4 Other Non-Functional Requirements

2.2.4.1 Performance Requirements

Requirement	Target
GPS simulation tick rate	1 packet per 3 seconds
Anomaly score computation (client-side)	< 10ms per GPS packet
Anomaly score computation (FastAPI scorer)	< 200ms per request
Map render latency (geofence add/delete)	< 100ms
LSTM inference (single input)	< 50ms on CPU
Dashboard initial load time	< 3 seconds on 10 Mbps connection
ML pipeline training time (50 GeoLife files, 1 user)	< 5 minutes on standard laptop CPU

2.2.4.2 Safety Requirements

- Anomaly threshold (0.6) must not be modifiable by end users via the UI, to prevent accidental suppression of alerts.
- Alert banners must persist on screen until explicitly dismissed auto-dismissal is not permitted.
- If the SignalR connection drops, the dashboard must fall back to Local Storage-based local scoring without interrupting simulation. A connection status indicator must inform the user.
- All GPS coordinates handled by the system are simulated; no real personal location data is processed in the prototype. If the system is extended to real devices, data must be transmitted over HTTPS only.

2.2.4.3 Security Requirements

- Backend API endpoints must validate input coordinates (lat in [-90, 90], lon in [-180, 180]) and reject malformed requests with HTTP 400.
- PostgreSQL credentials must not be hardcoded; must be injected via environment variables in Docker Compose.
- No user PII is collected or stored in the current prototype scope.
- CORS policy on the .NET API must restrict origins to the configured dashboard domain only in production.
- Local Storage data is not encrypted; this is acceptable for prototype/simulation data. Any production deployment handling real GPS data must use a server-side session with HTTPS.

2.3 Cost Analysis

Item	Category	Estimated Cost (₹)	Notes
ESP32 Dev Board	Hardware	450	Robu.in; qty 2 = ₹900
Neo-6M GPS Module	Hardware	350	Standard UART GPS
SIM800L GSM Module	Hardware	600	Includes SIM slot
MPU-6050 IMU	Hardware	80	I^2C accelerometer/gyro
Miscellaneous (wires, PCB, housing)	Hardware	500	Breadboard proto stage
Hardware Subtotal		**₹2,430**	
Cloud VM (Azure / AWS)	Infrastructure	₹0	Free tier used for dev; est. ₹2,000/month production
Domain / SSL	Infrastructure	₹800/year	If deployed publicly
GeoLife Dataset	Data	₹0	Microsoft public dataset
Python / PyTorch / React	Software	₹0	Open source
.NET 8 SDK	Software	₹0	Open source
Docker Desktop	Software	₹0	Free for personal use
Total Project Cost (Prototype)		**~₹3,230**	Excluding cloud production costs

2.4 Risk Analysis

Risk	Probability	Impact	Mitigation
LSTM overfitting on single GeoLife user (user 000)	High	High	Train on multiple users; evaluate on held-out test sequences; add dropout regularization
Anomaly threshold (0.6) generates excessive false positives for irregular routines	Medium	High	Make threshold configurable per deployment; evaluate F1 score on validation data
Leaflet tile loading failure (offline use)	Medium	Low	Tiles are cosmetic; geofence logic and simulation run entirely client-side regardless
localStorage quota exceeded with many geofences	Low	Medium	Cap stored geofences at 50; migrate to IndexedDB for larger datasets

SignalR connection instability in poor network	Medium	Medium	Implemented fallback to localStorage mode; user is notified via status indicator
GeoLife data quality issues (gaps, outliers in trajectories)	Medium	Medium	Preprocessing pipeline validates timestamps and filters unrealistic speeds (>200 km/h)
Docker Compose setup failure in evaluation environment	Medium	Low	Dashboard operates fully standalone without Docker; documented in README
GSM/GPS hardware unavailability	Low	Low	Current prototype is software-only; hardware is future-phase scope

Chapter 3 - Methodology Adopted

3.1 Investigative Techniques

3.1.1 Nature of the Investigation

The methodology followed for CareTag does not fit cleanly into a single classical investigative category. The project neither tests a hypothesis on a live human cohort, as a purely experimental study on human subjects would, nor does it merely describe an existing phenomenon without intervention, as a purely descriptive study would. Instead, the team adopted a combined descriptive-experimental investigation, in which a descriptive analysis of real-world GPS trajectory data is used to characterise what constitutes ordinary, routine movement, and an experimental/developmental investigation is then used to build, exercise and evaluate the algorithmic pipeline that converts this characterisation into a working anomaly-detection and alerting system.

This combination was chosen deliberately rather than by default. A purely comparative investigation, in which CareTag would be evaluated against a control group of users monitored with a conventional tracker, was not feasible at the mid-semester stage because it requires live deployment on human participants, ethical clearance, and a monitoring period long enough for both routines and genuine anomalies to emerge. A purely experimental investigation was also not sufficient on its own because, before any deviation can be experimentally detected, the system first needs a descriptive model of what "normal" looks like for a given individual. The descriptive component therefore precedes and feeds the experimental component: historical trajectories are studied to extract recurring stay locations and time-of-day patterns, and only once this descriptive routine model exists does the investigation become experimental in nature, through controlled injection of movement that departs from the learned routine and observation of how the system responds.

3.1.2 Application to the CareTag Pipeline

The investigative approach is applied consistently across the full processing pipeline of CareTag: raw GPS trajectories are first cleaned and structured, meaningful stay points are extracted from the continuous trajectory, these stay points are grouped into representative locations using clustering, a sequence model is trained on the resulting location-and-time sequence to learn the individual's routine, incoming positions are compared against this learned routine to compute a deviation score, the same position is checked against caregiver-defined geofences, and the combined outcome is used to decide whether an alert should be raised and shown on the caregiver dashboard. Each stage of this pipeline is treated as a distinct unit of investigation with its own observable inputs and outputs, which keeps the evaluation of the system traceable rather than treating the pipeline as an unexaminable black box.

The data observed and recorded at each stage include the raw latitude, longitude and timestamp triples that make up a trajectory; the derived stay points, each associated with an arrival time, a departure time and a dwell duration; the cluster label assigned to each stay point once clustering has been performed; the time-slot representation used to index a user's day (for example, dividing the day into fixed intervals such as morning, afternoon, evening and night, or finer hourly bins, depending on the granularity implemented); the sequence of recent cluster visits fed into the routine model; the routine model's output, expressed as an expected next location or a probability distribution over the known clusters for the current time slot; and, finally, the spatial and temporal deviation values computed by comparing the actual observed position against this expectation.

Spatial features studied in this investigation are principally the geographic coordinates of stay points and cluster centroids, together with the distance between an observed position and the nearest expected cluster centroid, computed using the Haversine formula so that the curvature of the earth is correctly accounted for over the short distances typical of daily movement. Temporal features studied include the time of day, the day of the week, and the position of an event within the discretised time-slot scheme used by the routine model. Sequential behaviour is studied through the ordered history of visited locations that the LSTM-based model consumes to anticipate the next probable location; this sequential view is what allows CareTag to move beyond a memoryless, point-in-time check and instead reason about whether the current movement is consistent with the pattern of movement that has led up to it.

3.1.3 Evaluation Criteria and Success Measures

Because the publicly available trajectory data used to develop and exercise the pipeline records ordinary commuting behaviour and does not contain labelled instances of the kind of behavioural anomaly CareTag is intended to catch, the evaluation strategy could not rely on conventional supervised metrics computed against a labelled anomaly set. Instead, three complementary forms of evaluation were used. First, the clustering stage was evaluated descriptively, by inspecting the

number and compactness of the clusters produced for a given value of K and confirming that they correspond to visually and semantically distinct stay locations rather than to fragments of the same place; this is an observational check rather than a statistical hypothesis test. Second, the routine-modelling stage was evaluated experimentally by holding out the later portion of each trajectory's time-ordered sequence and measuring how often the model's expected-location output matches the location the user actually visited next, which gives a next-location prediction accuracy that indicates whether the model has genuinely captured recurring structure rather than memorising the training sequence. Third, the anomaly-scoring, geofencing and alert-dispatch stages were evaluated through controlled scenario testing using the project's GPS simulation module: a simulated device is made to replay a trajectory consistent with the learned routine, and the resulting deviation score is observed to remain low, after which the same device is made to depart from the routine, for instance by remaining stationary outside a known cluster during a time slot in which movement is expected, or by entering a location that has never been visited in that time slot, and the deviation score and any resulting alert are observed to confirm that the system responds appropriately.

A CareTag prototype run is considered successful, for the purposes of this stage of the project, when stay points are correctly extracted from a raw trajectory without discarding genuine dwell locations; when the clusters produced are stable and interpretable; when the routine model's next-location predictions are noticeably better than a naive baseline that always predicts the most frequently visited cluster; when replay of a routine-consistent trajectory keeps the combined deviation score below the configured alert threshold throughout; when a deliberately injected deviation causes the score to cross the threshold and an alert to be generated and displayed on the caregiver dashboard without requiring a page refresh; and when a geofence violation, tested independently of the behavioural model, is flagged correctly regardless of whether the behavioural component also flags the same event. Comparing the behavioural, routine-aware approach with traditional static geofencing is treated at this stage as a conceptual and feature-level comparison rather than a field trial: the two approaches are examined side by side in terms of what each can and cannot detect (for example, a static geofence cannot flag a user who never leaves a safe zone but stops moving at an unusual time, whereas the behavioural layer can), and this comparison is used to justify the design rather than to claim a measured improvement over a competing product.

3.1.4 Limitations of the Current Investigative Setup

Several limitations of the present investigative approach are acknowledged. The routine model is currently trained and validated using the publicly available Microsoft GeoLife GPS trajectory dataset, which was collected from a general population of contributors rather than from school children, elderly individuals or patients with cognitive impairments; it is used at this stage as a realistic stand-in for personal trajectory data because no live deployment data from the project's actual target users yet exists. The investigation also inherits the assumption, stated at the project-planning stage, that target users follow reasonably fixed daily routines that a sequence model can learn; users whose routines vary substantially from day to day would be expected to produce a

higher baseline deviation score, and this has not yet been separately quantified. The anomaly threshold used to decide when a deviation score constitutes an alert has, at this stage, been set through empirical observation of scores produced during simulated replay rather than through a statistically derived operating point calibrated against a large, labelled set of genuine anomalies, since no such labelled set is available. Finally, all evaluation performed so far is simulation-based, using software-generated GPS streams rather than a live embedded device and a live caregiver in the field; consequently, real-world figures such as false-alarm rate, alert latency under production network conditions, or caregiver acceptance of the system have not yet been measured and are identified as evaluation work for a later phase of the project.

3.2 Proposed Solution

3.2.1 Overall Solution Concept

The central idea behind CareTag is to shift safety monitoring away from a single binary boundary check and toward a layered, personalised model of expected behaviour. A traditional static geofence only asks whether a device is inside or outside a fixed area, and therefore treats every location inside the boundary as equally safe and every location outside it as equally dangerous, regardless of what the user is actually doing. CareTag retains this boundary check, because it remains a simple and dependable safeguard, but adds a second, behavioural layer above it. This behavioural layer builds a routine-aware model of where a specific individual is normally expected to be at a given time of day, and continuously compares live or simulated position data against that expectation to produce a deviation score. The result is a monitoring system that can respond both to conventional boundary violations and to subtler forms of risk, such as being static in an unusual place during a period when the user is normally active, or moving through an unfamiliar sequence of locations that does not resemble any previously learned routine, even while the user technically remains inside the caregiver-defined safe zone.

3.2.2 Data Acquisition

At the current prototype stage, the routine-learning pipeline is developed and validated using the Microsoft GeoLife GPS trajectory dataset, a large collection of real, timestamped latitude-longitude-altitude trajectories originally gathered from volunteer users over an extended period. This dataset is used as a practical substitute for genuine per-user CareTag device data because it exhibits the same underlying structure the system is designed to work with, repeated visits to a limited set of locations, recognisable daily and weekly patterns, and periods of both movement and prolonged dwell, while being immediately available for development and testing without requiring live hardware. In production use, the same processing pipeline is intended to consume position data streamed from an individual's own CareTag device (or, at this software stage, from the project's GPS simulation module), so that the routine model is trained specifically on that individual's history rather than on generic population data.

3.2.3 Data Preprocessing and Stay-Point Extraction

Raw trajectory records are first parsed into a uniform structure of latitude, longitude and timestamp, and records that are clearly invalid, for instance points with missing coordinates, duplicate timestamps, or gaps that indicate a lost GPS fix, are filtered out before further processing. Because a continuous GPS trace produces far more points than are actually useful for understanding behaviour, most of them simply describing straight-line travel between two places, the cleaned trajectory is reduced to a smaller set of stay points. A stay point is identified whenever the user remains within a defined spatial radius for at least a defined minimum duration, and it is recorded together with its arrival time, departure time and computed dwell duration. This conversion from a dense stream of coordinates into a sparse set of meaningful stay locations is what makes the remainder of the pipeline computationally tractable and behaviourally interpretable: routine modelling operates on "where the user stopped and for how long" rather than on the far noisier question of the exact path taken between two stops.

3.2.4 Location Clustering

The stay points extracted across an individual's history typically correspond to the same handful of physical places, home, school or workplace, a relative's residence, a park, and so on, visited repeatedly over time. To turn these repeated visits into a small, stable set of representative locations, CareTag applies K-Means clustering to the coordinates of the extracted stay points. Each resulting cluster centroid stands in for one recognisable place in the user's routine, and every subsequent stay point is associated with the nearest cluster rather than with its raw coordinates, which absorbs the small GPS positioning noise that would otherwise make two visits to the same place look like two different locations. The number of clusters, K, is selected by observing how the within-cluster compactness changes as K increases and choosing a value beyond which additional clusters stop producing meaningfully separated groups, so that the clusters correspond to genuinely distinct places rather than to arbitrary subdivisions of a single location. These clustered locations become the vocabulary of "places" that the routine model reasons over in the next stage.

3.2.5 Routine and Behaviour Modelling

Once stay points have been reduced to a small set of clustered locations, CareTag learns the individual's routine using a sequential model built around a Long Short-Term Memory (LSTM) network. Each stay point is represented by the identifier of its cluster together with a time-slot indicator describing when in the day it occurred, and the ordered history of recent stay points is fed into the LSTM so that the model can capture dependencies between consecutive locations rather than treating each visit in isolation. Given a recent window of visited locations and the current time slot, the model produces an expected next location, expressed either as a single most-likely cluster or as a probability distribution over the known clusters. An LSTM was chosen over a simpler frequency-based lookup precisely because daily routines are sequential rather than purely time-indexed: where a user goes next often depends on where they have just been, and a model that only

asked "where does this user usually go at 5 p.m." would miss that dependency. At the same time, the model's capability is not overstated within this report, it is trained on a limited window of history, it predicts a coarse cluster rather than an exact coordinate, and its accuracy is bounded by how consistent the underlying routine actually is.

3.2.6 Anomaly Detection

Behavioural deviation in CareTag is computed from two complementary components rather than from a single measurement. The spatial component compares the user's actual current position with the centroid of the location the routine model expected them to be at, using the Haversine formula to obtain a real-world distance in metres, which is then normalised so that it can be combined with other quantities on a common scale. The temporal component is derived from the LSTM's own output, specifically the probability the model assigns to the cluster in which the user is actually found for the current time slot; a low assigned probability indicates that the observed location is temporally unusual for that point in the day, even if it is a place the user has visited before. These two components are combined, using a weighted formula, into a single continuous deviation score rather than a hard pass/fail flag, and an alert is raised only once this combined score crosses a configured anomaly threshold. In practical caregiver-facing terms, the deviation score functions as a running indicator of how closely the present moment matches the individual's established pattern of life: a score near the low end of the scale means the user's current location and timing look ordinary, while a score approaching or exceeding the threshold indicates that either the place, the timing, or both, no longer resemble anything the model has previously learned for that person.

3.2.7 Geofence Module

Independently of the behavioural layer, CareTag retains a conventional geofencing module through which a caregiver can define one or more safe-zone boundaries directly on the dashboard map. Each incoming position is checked against every active geofence, and a boundary violation is flagged the moment the position falls outside all configured safe zones. This geofence check and the behavioural deviation check are deliberately kept as two separate, parallel evaluations rather than being merged into a single score: a geofence violation is raised on its own regardless of what the routine model thinks, and a behavioural anomaly is raised on its own regardless of whether the user is inside or outside a safe zone. The two are only brought together at the point of alert generation, where the caregiver dashboard labels each alert with its originating cause, geofence violation, behavioural deviation, or both simultaneously, so that a caregiver reviewing an alert can immediately tell which kind of risk triggered it.

3.2.8 GPS Simulation Environment

Because the current prototype is validated in software ahead of live hardware deployment, CareTag includes a GPS simulation module that stands in for a physical device during development and testing. The simulator generates a stream of position updates that mimics the behaviour of a real CareTag unit, moving a virtual device through defined movement phases, for example travelling

toward a location, dwelling there, and travelling onward, and periodically emitting telemetry packets containing position, a small amount of realistic positional jitter, an approximate speed, and a simulated battery level. This simulated telemetry is fed through the same preprocessing, clustering, routine-comparison and geofence-checking logic that would be used with real device data, which allows the full anomaly-alert pipeline, from raw position to caregiver-visible alert, to be exercised end to end without requiring a working hardware unit. The simulation environment is treated explicitly as a development and testing aid, standing in for the physical embedded device described in the project's hardware plan rather than as a permanent replacement for it.

3.2.9 Caregiver Dashboard

The caregiver-facing side of CareTag is a React-based web dashboard that presents the output of the pipeline in a form a non-technical caregiver can act on immediately. A Leaflet-based interactive map forms the centre of the dashboard, showing the monitored individual's current or last-known location together with any configured geofence boundaries, which the caregiver can draw and edit directly on the map. Alongside the map, the dashboard surfaces the current behavioural deviation score, a short activity log summarising recent stay points and movement, and a list of active and past alerts distinguishing geofence violations from behavioural anomalies. The current prototype is built to operate offline-first, storing recent state in the browser's local storage so that the dashboard remains usable and continues to display the last known status even when a live connection to a backend service is temporarily unavailable, which reduces the system's dependence on constant connectivity and keeps the software prototype fully functional on its own during development and demonstration.

3.2.10 Backend and Deployment Architecture

The processing pipeline described above, preprocessing, clustering, routine modelling and anomaly scoring, is implemented as a Python-based service and can run either directly alongside the dashboard for local, offline-first operation, or behind an optional backend for multi-device, multi-caregiver deployment. Where a backend is used, the current plan is to expose the ML pipeline through a lightweight FastAPI service and to persist device, geofence, alert and routine data in a PostgreSQL database; a separate .NET 8 service layer, communicating alerts to connected dashboards in real time through SignalR, is planned as the production-grade API and notification layer for a scaled, multi-user deployment. It is important to state clearly that this backend architecture is, at the present stage of the project, a planned and partially scaffolded component rather than a fully deployed production system: the software prototype demonstrated so far runs the pipeline and the offline-first dashboard as a self-contained unit, and the FastAPI/.NET/PostgreSQL/SignalR arrangement represents the intended path for taking that prototype into a deployable, multi-caregiver product rather than a claim that such a deployment is already live.

3.2.11 End-to-End Workflow

The complete flow of information through CareTag, from raw position data to a caregiver-visible outcome, proceeds as follows:

1. GPS trajectory data is obtained, either from the GeoLife dataset during development, from the GPS simulation module during testing, or from a live device in eventual field deployment.

2. The incoming data is preprocessed, parsed, timestamp-normalised, and filtered to remove invalid or duplicate readings.

3. Stay points are extracted from the cleaned trajectory based on spatial and temporal dwell thresholds.

4. Stay points are clustered using K-Means to obtain a stable set of representative locations.

5. The LSTM-based routine model is trained on, or queried with, the resulting sequence of clustered locations and time slots.

6. A new position is received for evaluation, either from the simulator or, eventually, from the live device.

7. The routine model determines the expected location and time-slot probability for the current moment.

8. The spatial deviation (Haversine distance from the expected location) and the temporal deviation (routine-model probability of the observed location) are computed and combined into a single deviation score.

9. The same position is independently checked against all active caregiver-defined geofences.

10. An alert condition is determined based on whether the deviation score exceeds the anomaly threshold, whether a geofence has been violated, or both.

11. Any resulting alert, together with live location, activity summary and deviation score, is pushed to and displayed on the caregiver dashboard, with local storage preserving the last known state for offline-first viewing.

12. Where the optional backend is enabled, all relevant data, device readings, computed scores, geofence configuration and generated alerts, is additionally persisted to the PostgreSQL database for later retrieval and audit.

3.3 Work Breakdown Structure

The Work Breakdown Structure adopted for CareTag decomposes the project into ten major work packages, carried forward from the project's original planning documentation and re-organised here to reflect the status actually reached by the mid-semester stage. Work packages that correspond to the software prototype described in Section 3.2 are labelled as current implementation, while work packages that depend on physical hardware and field deployment are labelled as planned or future implementation, consistent with the fact that the prototype demonstrated so far is a software system exercised through the GPS simulation module rather than a deployed embedded device.

WBS ID	Work Package	Status	Owner(s)	Key Activities / Output
1.0	Project Planning & Requirement Analysis	Completed	Manya, Guneev, Kinjal, Bhavishya, Anureet	Problem statement, need analysis, objectives and assumptions/constraints finalised; approved project proposal.
2.0	Literature Survey	Completed	All members	Review of existing trend-detection, geofencing and wearable-safety literature informing the design choices in 3.2.
3.0	Data Acquisition & Preprocessing	Current (software prototype)	Kinjal, Guneev	GeoLife trajectory ingestion, cleaning and stay-point extraction pipeline.
4.0	AI/ML Development (Clustering, Routine Model, Anomaly Scoring)	Current (software prototype)	Kinjal	K-Means location clustering, LSTM routine model, combined deviation-score logic.
5.0	Context-Aware Geofencing & GPS Simulation Logic	Current (software prototype)	Guneev, Anureet	Geofence violation checks and the GPS simulation module used for pipeline testing.
6.0	Caregiver Dashboard (Frontend)	Current (software prototype)	Manya	React and Leaflet based dashboard with offline-first, local-storage operation, alerts and activity log.
7.0	Backend / Cloud Integration	Planned / partially scaffolded	Guneev	Optional FastAPI service, PostgreSQL storage, .NET 8 API layer and SignalR-based real-time alert delivery.
8.0	Hardware Selection & Embedded Integration	Planned / future implementation	Anureet, Bhavishya	MCU, IMU, GPS and GSM module selection and firmware-level integration for the physical wearable device.
9.0	Testing & Optimisation	Ongoing	Bhavishya, all members	Simulation-based pipeline testing, dashboard validation, and scenario-based anomaly/geofence evaluation.
10.0	Documentation & Final Submission	Ongoing	All members	Technical report, presentation material and final capstone documentation.

These work packages are not independent; each later package builds directly on the output of one or more earlier ones. Data acquisition and preprocessing (3.0) supplies the cleaned stay-point data that AI/ML development (4.0) depends on, since neither clustering nor routine modelling can be trained on raw, unfiltered trajectories. The output of AI/ML development, the trained clustering model, the routine model and the anomaly-scoring logic, is in turn what the geofencing and simulation logic (5.0) draws on to compute a combined deviation score during simulated playback, and it is this combined score, together with geofence status, that the caregiver dashboard (6.0) consumes to render alerts and the activity log. Backend and cloud integration (7.0) is designed to

sit alongside the existing pipeline and dashboard rather than to replace them, persisting the same data the dashboard already displays so that a scaled, multi-caregiver deployment becomes possible without redesigning the core algorithms. Hardware selection and embedded integration (8.0) is largely decoupled from the software packages at this stage, since the GPS simulation module allows packages 3.0 through 6.0 to be built and demonstrated independently of the physical device; once hardware integration is complete, the simulated telemetry source will be replaced by the live device without requiring changes to the downstream processing pipeline. Testing and optimisation (9.0) runs throughout the later work packages rather than as a single terminal phase, validating each stage, clustering quality, routine-model prediction accuracy, deviation-score behaviour, geofence correctness and dashboard responsiveness, as it becomes available, and documentation (10.0) is maintained continuously alongside development so that the final report accurately reflects the state of the system rather than being reconstructed after the fact.

3.4 Tools and Technology

The tools and technologies adopted for CareTag were selected to match the two halves of the current prototype, a Python-based machine-learning pipeline that turns raw trajectory data into a routine-aware deviation score, and a JavaScript-based dashboard that presents the resulting alerts and telemetry to a caregiver, together with a set of optional, planned components for scaling the system beyond a single offline-first instance.

3.4.1 Programming Languages and Core Libraries

React framework described under the frontend category, is used for the caregiver dashboard, keeping the presentation layer in the language best suited to interactive, in-browser map rendering. Python is used throughout the data-processing and machine-learning pipeline because of its mature ecosystem for numerical and trajectory data handling and its direct compatibility with the ML libraries described below. Pandas and NumPy are used for loading, cleaning and manipulating GeoLife trajectory records and the derived stay-point tables, since both provide efficient vectorised operations over the timestamped coordinate sequences that the pipeline works with at every stage. JavaScript, through the

3.4.2 Machine Learning

Scikit-learn's K-Means implementation is used for location clustering because it is a well-tested, efficient implementation of the algorithm and integrates directly with the pandas-based data structures already used elsewhere in the pipeline. PyTorch is used to implement and train the LSTM-based routine model, since it gives the team direct control over the sequence-model architecture, the loss function used for next-location prediction, and the training loop, which is necessary because the model's output is consumed as a probability distribution used directly in the deviation-score calculation rather than as a simple classification label. Pandas and NumPy, already introduced above, are also used within the ML stage itself for feature preparation, such as building the sequence

windows that are fed into the LSTM and computing the normalised Haversine distances used in the spatial deviation component.

3.4.3 Frontend / Dashboard

React is used to build the caregiver dashboard because its component model maps naturally onto the dashboard's structure of a map view, an alert list, an activity log and a geofence-editing panel, each of which can be developed and updated independently. Leaflet is used for the interactive map layer, rendering the user's current or last-known location, the movement trail derived from recent stay points, and caregiver-drawn geofence boundaries directly on an open mapping base layer. The dashboard's offline-first behaviour is implemented using the browser's local storage, allowing the interface to retain and display the last known device state, deviation score and alert list even when no live connection to a backend is available, which was a deliberate choice given that the current prototype is designed to work as a self-contained unit without requiring the optional backend to be running.

3.4.4 Backend (Optional / Planned)

For the planned multi-device, multi-caregiver deployment, FastAPI is intended to expose the Python-based ML pipeline as a lightweight HTTP service, since it allows the existing pandas/PyTorch/scikit-learn code to be served with minimal additional wrapping. PostgreSQL is intended as the persistent store for device records, geofence definitions, historical positions and generated alerts, chosen for its reliability as a relational database for structured, relationship-heavy data of this kind. A separate .NET 8 service is planned as the production API and integration layer that would sit in front of caregiver clients, with SignalR used for pushing alerts to connected dashboards in real time rather than requiring the dashboard to poll for updates. As noted in Section 3.2.10, this backend layer is a planned architecture for scaling the system rather than a component that is already fully deployed; the current prototype's dashboard and pipeline function correctly without it.

3.4.5 Data and Dataset

The Microsoft GeoLife GPS trajectory dataset is the primary data source used to develop, train and validate the routine-modelling and clustering stages of the pipeline at this point in the project, standing in for genuine per-user CareTag data until live devices are deployed. Alongside the dataset itself, the preprocessing utilities built in Python for parsing raw trajectory files, filtering invalid points and extracting stay points are considered part of this data layer, since they determine the quality of everything the clustering and routine-modelling stages subsequently learn from.

3.4.6 Development and Version Control

Git, together with GitHub for hosting, is used for source-code version control and for coordinating work across the five team members responsible for the dataset/preprocessing pipeline, the machine-learning components, the geofencing and simulation logic, the dashboard, and the hardware and

testing tracks respectively, allowing each of these tracks to be developed and reviewed independently before being integrated into the combined prototype described in Section 3.2.

Chapter 4 – Design Specification

4.1 System Architecture

CareTag follows a three-tier architecture separating data ingestion, ML processing, and the application layer. The system is designed to operate in two modes: standalone (frontend + localStorage only, no backend required) and full-stack (frontend + .NET API + PostgreSQL + FastAPI scorer, orchestrated via Docker Compose).

Tier 1 — Data Sources: The ML pipeline consumes the Microsoft GeoLife GPS Trajectory Dataset v1.3, comprising 182 users and 17,621 trajectories in `.plt` format. In a production extension (out of scope for current prototype), a live GPS device (ESP32 + Neo-6M) would transmit coordinates via GSM to the backend API.

Tier 2 — ML Engine: A four-stage Python pipeline processes raw trajectory data through stay point extraction (Li et al., 2008 algorithm), K-Means clustering, dual-input LSTM training, and real-time anomaly scoring. Trained artifacts (`.joblib`, `.pth`) are exported and consumed by the FastAPI microservice at runtime.

Tier 3 — Application Layer: The React 18 dashboard communicates with the .NET 8 REST API for geofence CRUD and receives real-time telemetry via SignalR WebSocket. The .NET API connects to PostgreSQL for persistence and delegates anomaly scoring requests to the FastAPI Python microservice which loads the trained LSTM and K-Means models.

Figure 4.1: CareTag three-tier system architecture

The dashed box indicates planned hardware integration (out of current prototype scope). Double arrows in the application layer indicate bidirectional REST communication.

Technology Stack Table

Layer	Component	Technology	Version
Data	GPS trajectory dataset	GeoLife v1.3 (.plt)	2012
ML	Stay point extraction	Python, pandas	3.10+
ML	Clustering	scikit-learn K-Means	Latest
ML	Sequence modeling	PyTorch LSTM	2.x
ML	Anomaly scoring	Python (Haversine + softmax)	—
Frontend	Dashboard	React, Leaflet	18, Latest
Backend	REST API	.NET Minimal API	8
Backend	Scoring microservice	FastAPI	Latest
Backend	Real-time push	SignalR (WebSocket)	.NET 8
Database	Persistent storage	PostgreSQL	15.x
Infra	Container orchestration	Docker Compose	Latest

4.2 Design Level Diagrams

4.2.1 Use Case Diagram

The system has one primary actor the **Caregiver** who interacts with the geofence dashboard. The ML pipeline has no direct user actor; it runs offline during training and is invoked at runtime by the FastAPI scorer.

Figure 4.2: CareTag use case diagram

The Caregiver interacts with four primary use cases. "Create geofence" is extended by "Delete geofence," and "Receive alert" includes "Dismiss alert" as a mandatory sub-action. "Start simulation" includes "View telemetry." "View simulation map" is a standalone read-only use case.

4.2.2 Sequence Diagram — Anomaly Alert Flow

The most critical runtime interaction: a simulated trajectory point triggers an anomaly score above threshold and propagates an alert to the dashboard.

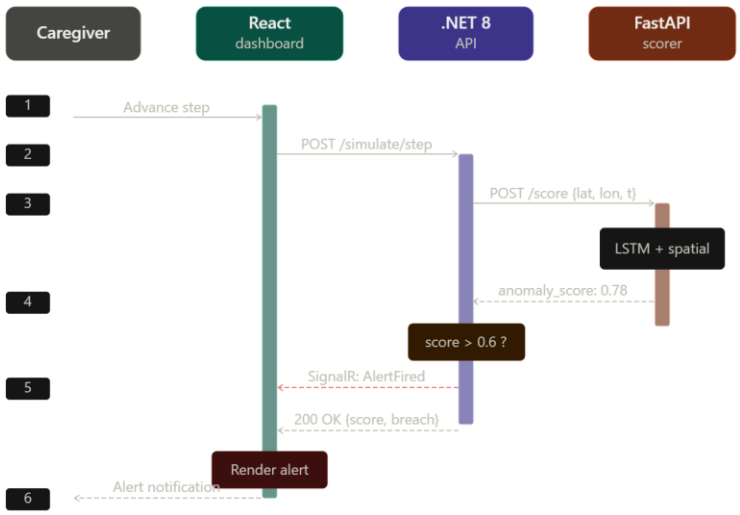


Figure 4.3: Sequence diagram — anomaly alert flow

Step 1: the caregiver clicks "Advance step" on the React dashboard. Step 2: the dashboard posts to the .NET API's /simulate/step endpoint. Step 3: the API forwards the GPS coordinate and time-slot to the FastAPI scorer via /score. Step 4: the FastAPI microservice runs the LSTM routine model and

computes a weighted anomaly score (returned as 0.78 in this example). Step 5: the .NET API compares the score against the 0.6 threshold; because it exceeds the threshold, it fires a SignalR AlertFired event over WebSocket. Step 6: the React dashboard receives the push event, renders the alert banner, and the caregiver is notified.

4.2.3 Activity Diagram — Geofence Breach Detection

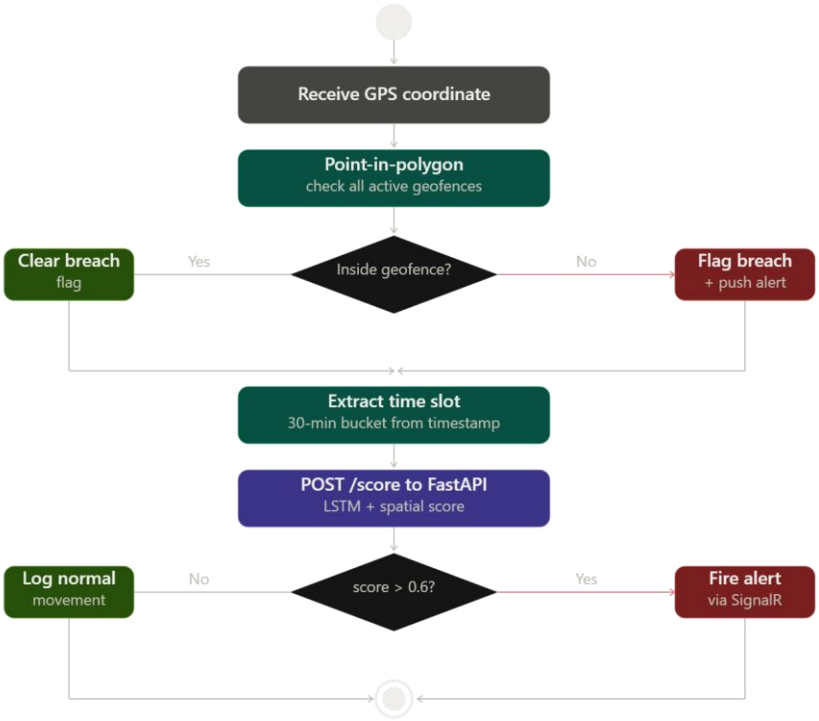


Figure 4.4: Activity diagram — geofence breach and anomaly detection

Each incoming GPS coordinate is processed through two parallel checks. First, a point-in-polygon test determines whether the position falls inside any active geofence; a breach immediately raises an alert and clears when the individual re-enters the zone. Second, regardless of geofence status, the coordinate and its 30-minute time-slot are forwarded to the FastAPI scorer; if the returned anomaly score exceeds 0.6, a second independent alert is fired via SignalR.

4.3 User Interface Diagrams

The dashboard is a single-page React application. Three principal screens are described below.

4.3.1 Geofence Management Screen

The main screen contains a full-width Leaflet map panel. A collapsible right-side panel hosts the geofence list and a "Draw geofence" button which switches the map into draw mode. Clicking the map in draw mode places polygon vertices; double-clicking closes the polygon and persists it to localStorage (offline) or PostgreSQL (full-stack). Each geofence row in the panel shows its name, colour swatch, and a trash icon to delete it.

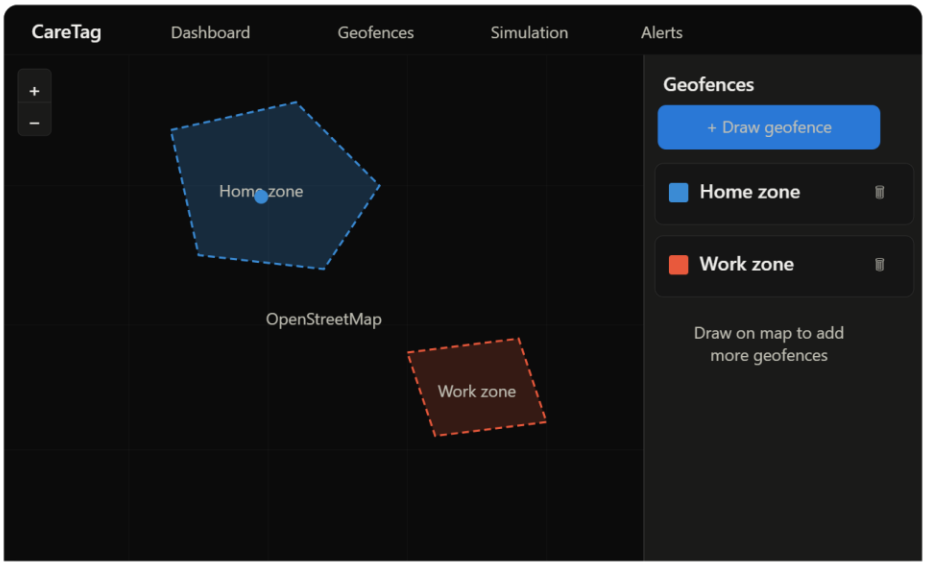


Figure 4.5: UI wireframe — geofence management screen

4.3.2 Simulation Screen

The simulation screen reuses the map with a trajectory line overlay. A control bar below the map exposes "Start," "Pause," and "Advance Step" buttons alongside a speed slider. A live telemetry panel on the right displays the current coordinate, time slot, anomaly score (with colour-coded severity), and geofence breach status for each step.

4.3.3 Alerts Screen

The alerts screen renders a chronological list of fired alerts. Each row shows the timestamp, the trigger type (geofence breach or anomaly), the anomaly score where applicable, and a "Dismiss" button. Dismissed alerts are retained in the log with a strikethrough style for audit purposes.

4.4 Snapshots of Working Prototype

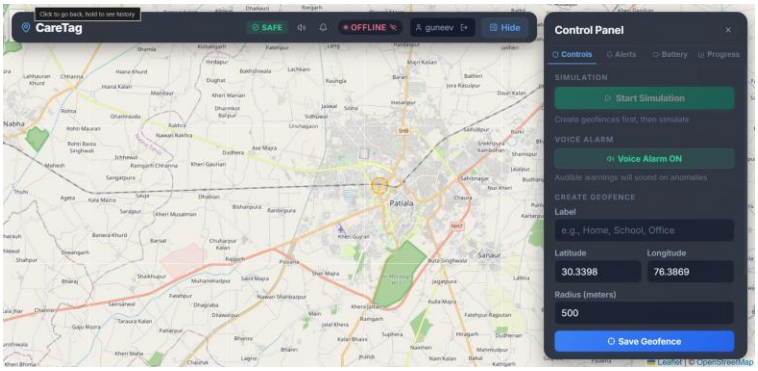


Figure 4.6 - Basic overview of CareTag

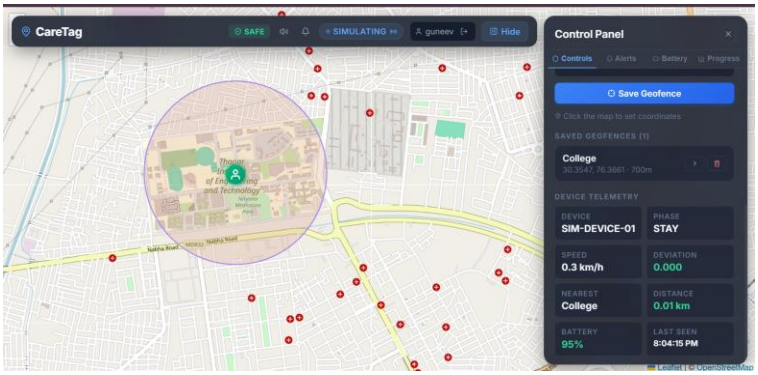


Figure 4.7 - Simulating the tracking of person

The following step-by-step walkthrough describes the working prototype in its current state. All screenshots below are taken from the standalone dashboard running at http://localhost:5173 with no backend dependency (localStorage mode).

Step 1 — Dashboard loads

On first load the map renders centred on Patiala (the GeoLife dataset location). The geofence panel on the right is empty with the prompt "Draw on map to add more geofences." The top navigation shows three tabs: Geofences, Simulation, and Alerts.

Step 2 — Caregiver draws a geofence

Clicking "Draw geofence" enters polygon draw mode. The cursor changes to a crosshair. The caregiver clicks five points on the map around a home address. Double-clicking the final point closes the polygon. A dialog prompts for a name and colour. On confirmation the geofence is saved to localStorage and appears both as a coloured dashed polygon on the map and as a named row in the geofence list panel.

Step 3 — Simulation is started

Switching to the Simulation tab loads the GeoLife trajectory for user 000. The trajectory is rendered as a grey polyline on the map. Pressing "Start" begins auto-advancing through the trajectory at the selected speed (1 step per second by default). The current position marker moves along the line. The telemetry panel updates in real time: coordinate, 30-minute time slot, computed anomaly score, and geofence status.

Step 4 — Anomaly detected

At a trajectory point that deviates from the trained routine — for example, a late-night displacement to an unvisited cluster — the anomaly score exceeds 0.6. The anomaly score readout turns red and the alert banner appears at the top of the screen with the message "Anomaly detected: score 0.78." The step is also logged in the Alerts tab.

Step 5 — Geofence breach detected

When the simulated position exits the "Home zone" polygon the breach status indicator in the telemetry panel changes from "Inside" to "Breach" and a second alert is logged. Simultaneously, the polygon stroke changes from dashed blue to solid red on the map.

Step 6 — Alert dismissed

Navigating to the Alerts tab the caregiver sees both alerts. Clicking "Dismiss" on each alert marks it as acknowledged. The alert rows are retained with a strikethrough and a "Dismissed" badge but are no longer surfaced as active notifications.

Step 7 — Full-stack mode (Docker)

Running docker compose up --build in geofence-dashboard/backend/ starts the .NET 8 API on port 5000, the FastAPI scorer on port 8000, and PostgreSQL on port 5432. Reloading the dashboard at http://localhost:5173 with the environment variable VITE_BACKEND_URL=http://localhost:5000 set switches the data layer from localStorage to the REST API. Geofences are now persisted in the geofences PostgreSQL table and anomaly scoring is handled by the FastAPI microservice rather than the browser-side stub.

5. CONCLUSIONS AND FUTURE SCOPE

5.1 Work Accomplished

The work completed during the current phase of the CareTag project addresses the major functional requirements of the proposed GPS-based behavioral monitoring system. The project has been developed as a research prototype combining a machine learning pipeline with an interactive geofence monitoring dashboard.

The following work has been accomplished:

1. **GPS Trajectory Processing:** A data-processing pipeline has been developed to process GPS trajectory data from the Microsoft GeoLife dataset. Stay points are extracted using spatial and temporal thresholds, reducing raw GPS trajectories into meaningful locations.
2. **Location Clustering:** Extracted stay points are grouped using the K-Means clustering algorithm to identify frequently visited locations and generate representative location clusters.
3. **Behavioral Routine Modeling:** A dual-input LSTM-based model has been designed to learn an individual's movement routine using time-slot information and recent location history. The model predicts the expected location for a particular time of day.
4. **Anomaly Detection and Scoring:** CareTag combines spatial deviation, calculated using the Haversine distance from the expected location, with temporal deviation obtained from the LSTM model. These signals are combined into a deviation score, with values above 0.6 treated as anomalous. This provides more information than a simple inside/outside geofence condition.
5. **Interactive Geofence Dashboard:** A browser-based dashboard has been developed using React and Leaflet. It allows caregivers to create, name, resize, and delete circular geofences on an interactive map.
6. **GPS Simulation:** A simulation engine has been implemented to represent the movement of a virtual device between defined locations. The simulation generates telemetry containing parameters such as movement phase, speed, jitter, and battery level, with GPS packets generated at three-second intervals.
7. **Real-Time Anomaly Alerts:** The dashboard displays telemetry information and generates alerts when anomalous movement is detected. The interface provides the current movement phase, speed, deviation score, nearest geofence, and distance information.
8. **Offline and Backend Support:** The system supports an offline operating mode using browser localStorage. An optional backend architecture has also been incorporated using .NET 8, PostgreSQL, FastAPI, and SignalR for persistent storage, anomaly scoring, and real-time telemetry communication.

Thus, the current phase has successfully established the core machine learning, geofencing, simulation, anomaly detection, and visualization components required for the CareTag prototype.

5.2 Conclusions

CareTag demonstrates the feasibility of combining GPS trajectory analysis, machine learning-based behavioral modeling, and geofencing to develop a more context-aware safety monitoring system.

Traditional geofencing systems primarily depend on manually defined geographic boundaries and generally generate alerts when a monitored device crosses those boundaries. Such systems do not consider whether a particular location is normal for a specific individual at a particular time. CareTag addresses this limitation by learning movement patterns from historical GPS data and incorporating temporal context into anomaly detection.

The developed prototype successfully integrates stay-point extraction, K-Means clustering, LSTM-based routine prediction, spatial and temporal anomaly scoring, interactive geofence management, and real-time simulation into a single system. The combination of spatial distance and behavioral probability provides a continuous deviation score rather than relying only on binary geofence alerts.

The current implementation also demonstrates that the proposed architecture can operate in a lightweight browser-based environment while supporting an optional backend for future deployment. The prototype therefore provides a suitable foundation for extending CareTag from a simulated research system into a real-world safety monitoring solution.

However, the present implementation should be considered a prototype rather than a production-ready safety system. The current version does not process live GPS hardware data, does not include physical GPS/GSM integration, and does not provide multi-user authentication.

5.3 Environmental, Economic and Social Benefits

5.3.1 Environmental Benefits

The environmental benefits of CareTag are primarily indirect. Since the current system is software-based and operates using simulated GPS data, it does not require continuous physical infrastructure during the development and testing phase. The browser-based and offline capabilities also reduce dependence on continuously running backend infrastructure for basic demonstrations and testing.

Furthermore, the prototype is built largely using open-source software and publicly available datasets, reducing the requirement for additional proprietary software resources. The current implementation uses technologies such as Python, PyTorch, React, .NET, and PostgreSQL.

5.3.2 Economic Benefits

CareTag has the potential to provide a relatively low-cost foundation for GPS-based safety monitoring. The software stack primarily uses open-source technologies, while the GeoLife dataset is publicly available. The estimated prototype cost documented for the project is approximately ₹3,230, excluding future production cloud costs.

The modular architecture also allows the system to be developed incrementally. A basic software-only version can be demonstrated without physical hardware, while GPS, GSM, and sensor components can be added later as the system moves towards deployment.

5.3.3 Social Benefits

The primary social benefit of CareTag is its potential contribution to safety monitoring for individuals who may require additional supervision, particularly children and elderly users.

Unlike purely static geofencing, the proposed system considers an individual's historical movement patterns and time-dependent behavior. This can help distinguish between routine deviations and potentially concerning movements. The system also provides caregivers with a quantified deviation score along with location and distance information, giving more context to an alert.

The project can therefore contribute towards more intelligent, context-aware safety monitoring while reducing unnecessary dependence on manually configured rules.

5.4 Future Work Plan

The next phase of CareTag will focus on converting the current software prototype into a more complete and realistic safety monitoring system. The major areas of future development are:

1. **Physical GPS Hardware Integration:** Integrate an ESP32 microcontroller with a Neo-6M GPS module to acquire real-time location data instead of relying on simulated GPS coordinates.
2. **GSM-Based Communication:** Integrate the SIM800L GSM module to transmit GPS telemetry from the physical device to the backend when Wi-Fi connectivity is unavailable.
3. **Motion and Fall Detection:** Integrate an MPU-6050 IMU to capture motion-related information and explore fall-detection capabilities as an additional safety feature. The current report identifies this component as a future hardware integration.
4. **Real-Time Device Communication:** Replace the current simulation layer with live device telemetry and connect the physical device to the backend anomaly-scoring pipeline.
5. **Improved ML Generalization:** Train and evaluate the LSTM model using multiple users and larger trajectory datasets to reduce the risk of overfitting to individual movement patterns. The current risk analysis specifically identifies training on a single GeoLife user as a high-impact risk.
6. **Threshold Optimization:** Evaluate the anomaly threshold using validation data and metrics such as precision, recall, and F1-score to reduce false positives and improve the reliability of alerts.
7. **Personalized Adaptive Models:** Develop mechanisms that allow the behavioral model to continuously adapt as an individual's routine changes over time.
8. **Authentication and Multi-User Support:** Introduce secure authentication and role-based access so that multiple caregivers and monitored users can be managed independently.
9. **Mobile and Notification Support:** Extend the system to mobile devices and integrate notifications for important anomaly events, enabling caregivers to receive alerts without continuously monitoring the dashboard.
10. **Security and Privacy Enhancements:** For a production deployment involving real GPS data, implement secure communication using HTTPS, encrypted storage, appropriate access controls, and stronger privacy mechanisms. The current prototype does not process real personal location data.
11. **Performance and Real-World Evaluation:** Evaluate the complete system using real-world GPS trajectories and measure anomaly detection accuracy, false-positive rate, latency, battery consumption, and communication reliability.

The long-term objective is to evolve CareTag from a software-based research prototype into a complete, low-cost, real-time behavioral safety monitoring platform capable of combining GPS location, movement sensing, personalized routine learning, and intelligent anomaly alerts.

APPENDIX A - REFRENCES

[1] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[2] Ji, Y., Wang, L., Wu, W., Shao, H., & Feng, Y. (2020). A method for LSTM-based trajectory modeling and abnormal trajectory detection. IEEE Access, 8, 104063–104073. https://doi.org/10.1109/ACCESS.2020.2997967

[3] Li, Q., Zheng, Y., Xie, X., Chen, Y., Liu, W., & Ma, W. Y. (2008). Mining user similarity based on location history. In Proceedings of the 16th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (pp. 1–10). ACM. https://doi.org/10.1145/1463434.1463477

[4] Liao, L., Patterson, D. J., Fox, D., & Kautz, H. (2007). Learning and inferring transportation routines. Artificial Intelligence, 171(5–6), 311–331. https://doi.org/10.1016/j.artint.2007.01.006

[5] Lin, Q., Liu, X., & Wang, W. (2018). GPS trajectories based personalized safe geofence for elders with dementia. In 2018 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (pp. 505–514). IEEE. https://doi.org/10.1109/SmartWorld.2018.00111

[6] Mazimpaka, J. D., & Timpf, S. (2016). Trajectory data mining: A review of methods and applications. Journal of Spatial Information Science, 13, 61–99. https://doi.org/10.5311/JOSIS.2016.13.263

[7] Zheng, Y., Li, Q., Chen, Y., Xie, X., & Ma, W. Y. (2008). Understanding mobility based on GPS data. In Proceedings of the 10th International Conference on Ubiquitous Computing (pp. 312–321). ACM. https://doi.org/10.1145/1409635.1409677

[8] Zheng, Y., Zhang, L., Xie, X., & Ma, W. Y. (2009). Mining interesting locations and travel sequences from GPS trajectories. In Proceedings of the 18th International Conference on World Wide Web (pp. 791–800). ACM. https://doi.org/10.1145/1526709.1526816

APPENDIX B: Plagiarism Report

Field	Detail
Document Title	CareTag — GPS Behavioral Monitoring & Geofence Dashboard
Submitted By	Guneev Taneja, Kinjal, Manya Sharma, Bhavishya Singla, Anureet
Institution	Thapar Institute of Engineering and Technology, Patiala
Submission Date	22/08/2026
Tool Used	Turnitin
Similarity Index	11%
Internet Sources	7%
Publications	3%
Student Papers	1%